SOFTWARE DEVELOPMENT PLAN • CONFIDENTIAL

Finamite GrowthOS

Final Development Plan, Cost Model & Delivery Roadmap

A realistic, implementation-oriented engineering plan covering scope, architecture, complexity, timeline, infrastructure, AI/RAG costs, risks, milestones, and acceptance criteria for the Finamite GrowthOS MVP and subsequent phases.

PREPARED & PRESENTED BY

Gaurav Sood

Developer

DOCUMENT DATE

August 27, 2026

VERSION

v1.0 — Final

Source of truth: the supplied Product & Technical Requirements Document (v1.0). Estimates are clearly marked as estimates, with assumptions stated explicitly.

Table of Contents

> **Purpose:** Give the development team a realistic, implementation-oriented plan for building the Finamite GrowthOS MVP and subsequent phases, including scope, architecture, complexity, timeline, infrastructure, AI/RAG costs, risks, milestones, and acceptance criteria.
>
> **Source of truth:** The supplied Product & Technical Requirements Document (v1.0). Do not silently add product scope that conflicts with that document. Where estimates are uncertain, mark them as estimates and state the assumptions.

---

1. Executive Summary

Finamite GrowthOS is an AI-powered operating system for service businesses. The core product loop is:

Diagnose → Recommend → Act → Track → Improve

The MVP must prove that business owners will repeatedly use an AI system that understands their business and tells them what to do next.

MVP technical milestone

Authentication → Business Onboarding → Dashboard → AI Advisor → Knowledge Base → Daily Score → Health Check → Action Plan → Assignment → KPI Tracking → Admin Panel

Do **not** build the complete ERP, CRM, WhatsApp automation, or complex integrations in the MVP.

Recommended technical foundation

- Backend: existing Node.js/TypeScript architecture where applicable
- Primary database: **PostgreSQL**
- Vector search: **pgvector**, introduced when semantic RAG is implemented
- Object/file storage: S3-compatible or managed object storage; do not rely on ephemeral local filesystem storage in production
- AI: provider abstraction so the model can be changed without rewriting the application
- API: REST or equivalent modular API
- Authentication: secure session/JWT architecture
- Authorization: RBAC + tenant/business-level isolation
- Background jobs: queue/worker architecture for document ingestion, embeddings, notifications, reports, and other long-running work
- Observability: structured logs, error tracking, audit logs, usage metrics

---

2. Scope and Product Principles

2.1 Core principle

The product should answer:

> **What requires attention today, why does it matter, and what should the owner do next?**

The application must not become a generic chatbot or feature-heavy ERP in the first release.

2.2 Core AI flow

```
User Question
    ↓
Intent Detection
    ↓
Relevant Business Context
    ↓
Knowledge Retrieval
    ↓
Finamite Methodology
    ↓
AI Response
    ↓
Recommended Action
    ↓
User Confirmation
    ↓
Assignment / Action / SOP / Checklist
    ↓
Completion / Evidence
    ↓
New Business Data
    ↓
Next Recommendation
```

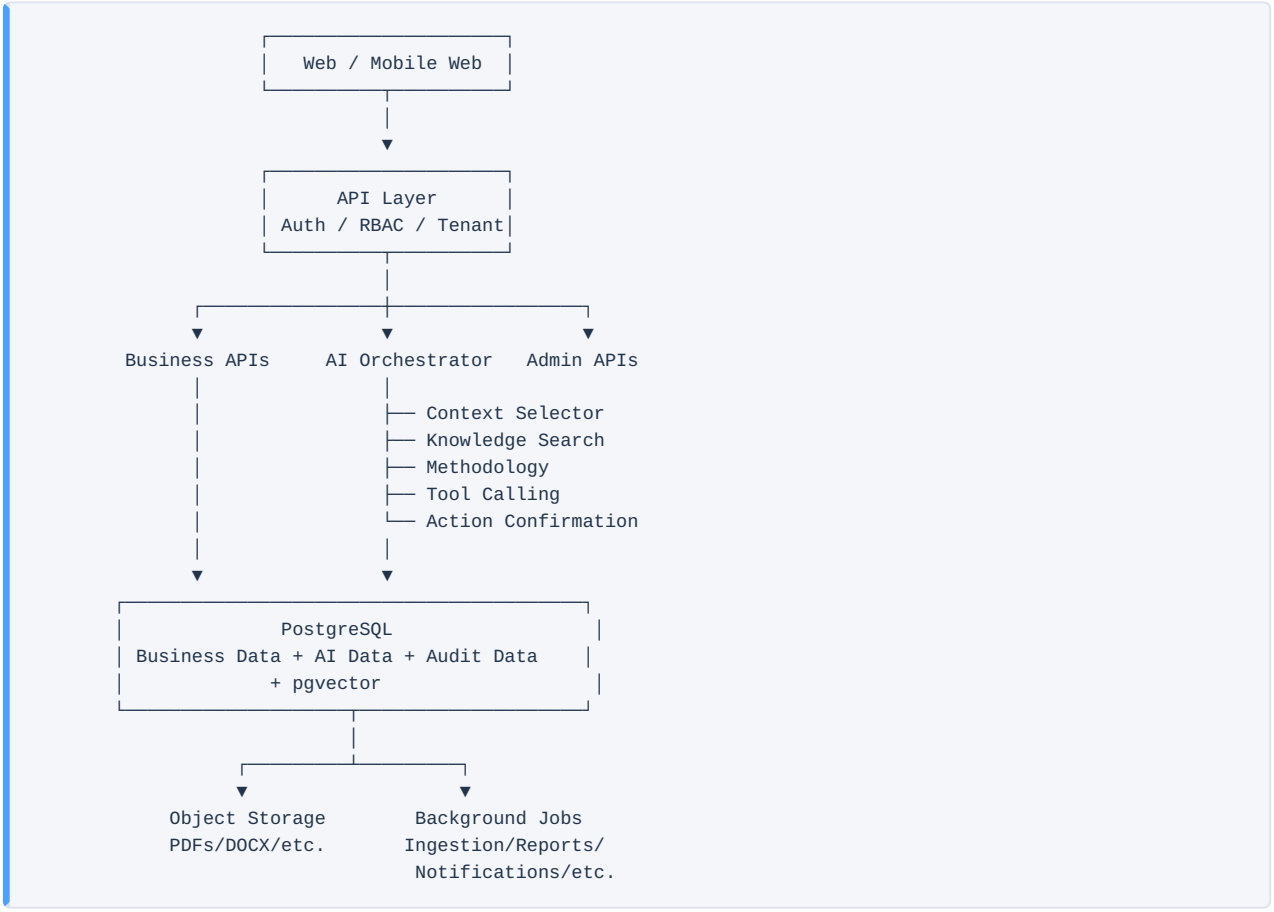
2.3 Non-negotiable architectural principles

1. Every business-owned record must be tenant-aware through `business_id`.
2. One business must never access another business's data.
3. Avoid sending the entire database to the AI.
4. AI context must be selected based on intent and relevance.
5. Proprietary Finamite methodology must have explicit priority.
6. AI must not invent unavailable business data.
7. AI-generated actions with side effects require user confirmation.
8. Subscription permissions must be configuration-driven, not scattered hardcoded checks.
9. Long-running work must not block HTTP requests.
10. Architecture must allow future team management, CRM, WhatsApp, ERP modules, and integrations.

---

3. Recommended Architecture

3.1 High-level architecture



3.2 PostgreSQL vs MongoDB

Recommended: PostgreSQL

PostgreSQL is the preferred primary database because the product has extensive relationships:

- users → businesses
- businesses → profiles/goals/KPIs
- assessments → questions → answers → reports
- action plans → action items → assignments
- subscriptions → plans → permissions
- AI conversations → messages → recommendations
- audit logs
- analytics

Use PostgreSQL JSON/JSONB where flexible AI metadata is needed.

pgvector

pgvector is **not an embedding model**.

- Embedding model/API: converts text into vectors.
- pgvector: stores vectors and performs similarity search.
- LLM: uses retrieved knowledge to generate the final answer.

Recommended RAG flow:

```
Document
  ↓
Text extraction
  ↓
Chunking
  ↓
Embedding model
  ↓
Vector
  ↓
PostgreSQL + pgvector

User question
  ↓
Embedding model
  ↓
Query vector
  ↓
pgvector similarity search
  ↓
Top relevant chunks
  ↓
LLM
```

Do not make pgvector mandatory for the first CRUD/business-data implementation. Add it when semantic knowledge retrieval is implemented.

---

4. MVP Scope

Priority 1 — Must Have

Authentication

- Signup/login
- Password security
- Session/JWT handling
- Role handling
- Account lifecycle

Business onboarding

- User profile
- Business profile
- Industry
- Business type
- Location
- Revenue
- Employees
- Clients
- Goals
- Business problems
- Existing systems

Home dashboard

- Business health
- KPI summary
- Today's priority
- Recommended action
- Recent activity

AI Advisor

- Conversation storage
- Intent detection
- Business context selection
- Knowledge retrieval
- Methodology retrieval
- Structured responses
- Tool/function calling
- Confirmation before side effects

Knowledge Base

- Admin upload
- PDF/DOCX/TXT/Markdown support
- Text extraction
- Chunking
- Metadata
- Embeddings
- Semantic retrieval
- Source attribution
- Status/version handling

Daily Business Score

- Configurable questions
- Daily check-in
- Configurable weights
- Score calculation
- Score history

Health Check

- Configurable questions
- Multiple question types
- Scoring
- Category scores
- AI report
- Historical reports

Action Plans

- Goal input
- AI-generated actions
- Action items
- Conversion to assignments

Assignments

- Assignment CRUD
- Priority
- Due date
- Status
- Completion %
- Evidence
- Comments
- Completion tracking

KPI Tracker

- Manual KPI entry
- Current value
- Target
- Previous period
- Change %
- Trend
- Status

Business Library

- Resources
- Categories
- Search
- Relevant resource recommendations

Admin Panel

- User management
- Business management
- Knowledge base management
- Assessment/question management
- Checklist/content management as required by MVP scope
- AI configuration
- Analytics
- Audit visibility

---

5. What NOT to Build in MVP

Explicitly defer:

- Full ERP
- Full CRM
- WhatsApp automation
- Email automation
- Calendar integration
- Advanced external integrations
- AI Meeting Assistant
- AI Decision Support
- Advanced team management
- Complex workflow automation

- Custom enterprise modules

These belong to later phases.

---

6. Database Design

6.1 Core entities

```
users
businesses
business_profiles
business_goals
business_kpis
kpi_entries
daily_checkins
daily_scores
health_assessments
assessment_questions
assessment_answers
health_reports
action_plans
action_items
assignments
assignment_submissions
sops
checklists
checklist_items
library_resources
knowledge_documents
knowledge_chunks
ai_conversations
ai_messages
ai_recommendations
notifications
streaks
badges
subscriptions
audit_logs
```

6.2 Tenant model

Every business-owned entity should include:

```
business_id
```

Where appropriate also include:

```
created_by
user_id
```

Authorization must verify the business relationship at the service/API layer. Do not rely solely on frontend filtering.

6.3 Future multi-business support

Do not assume:

```
User = Business
```

Use:

```
User
  ↓
Business Membership / Relationship
  ↓
Business
  ↓
Business Data
```

This avoids a future migration when users can manage multiple businesses.

---

7. AI Architecture

7.1 Context priority

The AI should prioritize:

1. User's business data
2. Finamite methodology
3. Relevant Finamite knowledge base
4. User history
5. General business reasoning

7.2 Context selection

Do not send all business data on every request.

Create an intent-aware context layer:

```
Intent
 ↓
Required context domains
 ↓
Fetch only required data
 ↓
Retrieve relevant knowledge
 ↓
Build context package
 ↓
LLM
```

Example:

```
Question: "Why are my employees missing tasks?"

Retrieve:
- team/business profile
- relevant assignments
- recent completion trends
- relevant SOPs
- delegation/accountability methodology
- relevant conversation history

Do not retrieve:
- unrelated marketing KPIs
- unrelated library resources
- unrelated financial records
```

7.3 AI tools

Implement tools conceptually similar to:

```
get_business_profile()
get_kpi_data()
get_health_score()
get_recent_assignments()
search_knowledge_base()
get_library_resource()
create_assignment()
create_action_plan()
create_sop()
create_checklist()
schedule_followup()
```

All side-effect tools must require appropriate authorization and explicit confirmation.

---

8. RAG / Knowledge Base Complexity

8.1 Pipeline

```
Upload
 ↓
Validate file
 ↓
Store original
 ↓
Extract text
 ↓
Normalize
 ↓
Chunk
 ↓
Attach metadata
 ↓
Generate embeddings
 ↓
Store chunks + vectors
 ↓
Index
 ↓
Search
```

8.2 Important metadata

```
title
category
subcategory
source
industry
business_stage
tags
status
created_at
updated_at
document_version
```

8.3 Retrieval

Use hybrid retrieval when mature:

```
Semantic vector similarity
        +
PostgreSQL full-text search
        +
Metadata filters
        ↓
Candidate ranking
        ↓
Top K chunks
        ↓
Context assembly
```

Do not depend exclusively on vector similarity forever. Metadata and keyword filtering are valuable for precision.

---

9. Subscription and Feature Architecture

Use feature permissions rather than scattered plan checks.

Example:

```
ai_advisor
daily_score
health_check
action_plans
assignments
kpi_tracker
sop_generator
team_management
crm
whatsapp
advanced_reports
```

Recommended conceptual structure:

```
Plan
 ├── Features
 ├── Limits
 └── Entitlements
```

Also support limits such as:

```
AI messages / month
knowledge uploads
team members
KPI count
storage
AI report generation
```

This is more scalable than:

```
if plan === "pro"
```

throughout the codebase.

---

10. Development Complexity

Use the following complexity scale:

- **S — Low:** straightforward CRUD/UI
- **M — Medium:** business logic + validation + integration
- **L — High:** multiple services, workflows, AI, asynchronous processing
- **XL — Very High:** cross-module orchestration, AI reliability, external integrations, significant testing

Module	Complexity	Primary risk
Auth	M	Security
Multi-tenancy	L	Data isolation
Onboarding	M	Schema flexibility
Dashboard	M	Aggregation/performance
AI Advisor	XL	Context/reliability/cost
Knowledge Base	XL	Parsing/RAG/retrieval
pgvector RAG	L	Retrieval quality
Daily Score	M	Configurable scoring
Health Assessment	L	Versioning/scoring
AI Health Report	L	Cost/quality
Action Plans	L	Structured generation
Assignments	M	Workflow/evidence
KPI Tracker	M	Data modeling
Library	M	Search/content
Admin Panel	L	Configuration breadth
Subscriptions	L	Entitlements/billing
Notifications	L	Scheduling/reliability
Analytics	M/L	Event design
Security hardening	L	Tenant isolation
QA	XL	Cross-module interactions

11. Realistic MVP Timeline

These are **development estimates**, not guaranteed delivery dates.

Solo experienced developer

Aggressive

10–14 weeks

Only realistic if:

- existing frontend/backend foundation is strong
- developer is experienced with Node/PostgreSQL/AI
- UI is not heavily customized
- admin tooling is kept lean
- minimal integrations
- extensive reuse of existing components

Realistic

16–24 weeks

Recommended planning assumption.

Conservative

24–32 weeks

Appropriate if:

- developer is learning parts of the stack
- AI/RAG quality requires substantial iteration
- requirements continue changing
- UI/UX is built from scratch
- QA is thorough

Small team

With 2–3 capable developers:

10–16 weeks is a realistic target for a usable MVP, assuming product decisions are finalized and work can run in parallel.

---

12. Suggested Sprint Plan

Sprint 1 — Foundation

1–2 weeks

- Repository/project setup
- Environment configuration
- PostgreSQL
- ORM/schema
- Auth
- RBAC
- Business/tenant architecture
- API conventions
- Error handling
- Logging

Sprint 2 — Onboarding

1–2 weeks

- User profile
- Business profile
- Goals
- Problems
- Systems
- Initial dashboard data model

Sprint 3 — AI Foundation

2–3 weeks

- AI provider abstraction
- Conversation model
- Intent layer
- Context selector
- Prompt architecture
- AI tools
- Token/cost tracking
- Safety/confirmation

Sprint 4 — Knowledge Base/RAG

2–3 weeks

- File upload
- Object storage
- Text extraction
- Chunking
- Embeddings
- pgvector
- Metadata filtering
- Retrieval
- Retrieval evaluation

Sprint 5 — Daily Score

1–2 weeks

- Questions
- Check-ins
- Scoring engine
- Configurable weights
- History
- Dashboard

Sprint 6 — Health Check

2–3 weeks

- Assessment engine
- Question management
- Answer storage
- Scoring
- AI report
- Historical reports

Sprint 7 — Actions & Assignments

2–3 weeks

- Action plans
- Action items
- Assignment CRUD

- Due dates
- Status
- Evidence
- Completion

Sprint 8 — KPI + Library

1–2 weeks

- KPI entry
- KPI dashboard
- Resource management
- Search
- Recommendations

Sprint 9 — Admin + QA

2–3 weeks

- Admin dashboard
- User/business management
- KB management
- Assessment configuration
- AI configuration
- Analytics
- Security review
- Performance testing
- End-to-end testing

Total realistic MVP

16–24 weeks for one strong developer

or approximately

10–16 weeks for a small capable team

---

13. Cost Model

All numbers below are **planning estimates**, not vendor quotes. Actual cost depends heavily on usage, region, traffic, model selection, storage, and architecture.

Use USD for baseline planning and convert to INR using the actual exchange rate when budgeting.

13.1 Development/testing phase

Lean testing

$20–$75/month

Suitable for:

- developer-only usage
- low AI volume
- small database
- limited document uploads

- low traffic

Heavy testing

$100–$300/month

Suitable for:

- repeated AI testing
- RAG experiments
- many conversations
- document ingestion
- background jobs
- staging environment

Extreme testing ceiling

$300–$500/month

This should be treated as a practical upper planning ceiling for a small MVP development environment unless deliberately stress-testing at high scale.

If costs exceed this during normal development, investigate:

- excessive AI context
- repeated report regeneration
- expensive model selection
- uncontrolled retries
- oversized prompts
- unnecessary embedding regeneration
- excessive logging/storage
- runaway background jobs

---

14. Cost Breakdown

14.1 Infrastructure

Potential components:

Component	Testing estimate/month
Backend hosting	$5–$30
PostgreSQL	$5–$40
Object storage	$1–$15
Background worker	$0–$30
Redis/queue	$0–$20
Monitoring/logging	$0–$30
Email	$0–$20
CDN/egress	$0–$20
Typical total	**$10–$150**

Avoid paying for separate infrastructure until traffic or reliability requirements justify it.

14.2 AI

AI is likely to become the largest variable cost.

Major cost drivers:

```
Number of AI requests
×
Input tokens
×
Output tokens
×
Model price
```

Additional factors:

- business context size
- RAG context size
- conversation history
- health report generation
- action-plan generation
- retry rate
- model tier
- embedding volume

Required engineering controls

Track per request:

```
user_id
business_id
model
input_tokens
output_tokens
estimated_cost
latency
intent
tools_used
retrieval_count
```

Then calculate:

```
AI cost / active business / month
AI cost / AI conversation
AI cost / report
AI cost / successful recommendation
```

---

15. Embedding/RAG Cost

Embedding generation is generally much cheaper than generation with a large LLM.

However, avoid repeatedly embedding unchanged documents.

Use:

```
document hash
+
chunk hash
+
embedding model/version
```

If unchanged:

do not regenerate the embedding.

Only re-embed when:

- content changes
- chunking strategy changes
- embedding model changes

---

16. Cost-Control Architecture

Implement these from the beginning:

16.1 Token budgets

Define limits for:

```
system prompt
business context
conversation history
RAG context
output
```

16.2 Context compression

Do not keep sending the entire conversation.

Use:

- recent messages
- conversation summary
- relevant historical messages

16.3 Retrieval limits

Start with a small top-K:

```
top_k = 5–10
```

and tune using evaluation data.

16.4 Model routing

Not every request requires the most expensive model.

Use different model tiers for:

- classification
- summarization
- retrieval assistance
- normal advice
- complex reasoning

- report generation

16.5 Caching

Cache:

- embeddings
- stable knowledge retrieval results where appropriate
- static methodology context
- generated summaries where safe

16.6 Rate limiting

Per:

```
user
business
IP
plan
```

16.7 Spending limits

Set provider-level usage limits and application-level budgets.

---

17. Production Cost Scenarios

These are broad planning scenarios, not guaranteed prices.

Scenario A — 100 active businesses

Assume:

- modest AI usage
- manual KPI entry
- limited documents
- no WhatsApp
- no large external integrations

Planning range:

$100–$500/month

Scenario B — 1,000 active businesses

Assume:

- regular AI Advisor usage
- daily check-ins
- RAG
- reports
- moderate document storage

Planning range:

$500–$2,500/month

Scenario C — 10,000 active businesses

Assume:

- regular AI use
- substantial RAG
- background processing
- analytics
- notifications
- team features

Planning range:

$3,000–$15,000+/month

At this point, cost optimization and infrastructure architecture become major engineering concerns.

---

18. Complexity of AI Cost

Do not estimate AI cost only by number of users.

Use:

```
Active users
×
AI requests/user
×
average input tokens
×
average output tokens
```

And separately:

```
documents
×
average chunks/document
×
embedding cost
```

And:

```
health reports
×
report token cost
```

Create a spreadsheet/model with adjustable assumptions instead of hardcoding a single monthly number.

---

19. Phase 2

Build only after the MVP proves usage.

Features

- AI SOP Generator

- Checklist engine
- Growth Roadmap
- Notifications
- Streaks
- Badges
- stronger automation
- better AI follow-up

Estimated duration

6–12 weeks

depending on how much of the MVP foundation is reusable.

Complexity

Medium → High

The main challenge becomes cross-module automation and reliable triggers.

---

20. Phase 3

The requirements identify:

- Team Management
- AI Meeting Assistant
- AI Decision Support
- CRM
- Advanced reporting
- WhatsApp
- Email
- Calendar integration

Estimated duration

3–6 months

for a small team, depending heavily on external integrations.

Complexity

High → XL

External APIs, permissions, background jobs, synchronization, failure recovery, rate limits, and user consent become major concerns.

---

21. Phase 4 — Full Operating System

Potential modules:

- ERP
- Finance
- HR

- Operations
- Advanced CRM
- Workflow automation
- Custom modules
- External integrations

Estimated duration

6–18+ months

This should not be treated as one project. Break it into independent products/modules.

Complexity:

XL

---

22. Overall Product Timeline

A realistic planning model:

```
MVP
16–24 weeks
       ↓
Phase 2
6–12 weeks
       ↓
Phase 3
3–6 months
       ↓
Phase 4
6–18+ months
```

For a small team, these can overlap once the core architecture stabilizes.

---

23. Biggest Technical Risks

23.1 AI context quality — CRITICAL

Bad context produces bad recommendations.

Mitigation:

- intent classification
- context selection
- retrieval evaluation
- source ranking
- methodology priority
- conversation summarization

23.2 RAG quality — CRITICAL

The system may retrieve irrelevant knowledge.

Mitigation:

- hybrid search
- metadata filters
- chunk-quality evaluation
- top-K tuning
- reranking if needed
- retrieval evaluation dataset

23.3 Tenant isolation — CRITICAL

A business must never see another business's data.

Mitigation:

- service-level authorization
- tenant-scoped queries
- integration tests
- authorization middleware
- database constraints/indexes
- security tests

23.4 AI cost explosion — HIGH

Mitigation:

- token budgets
- model routing
- context limits
- caching
- rate limits
- usage tracking
- provider spending limits

23.5 Long-running jobs — HIGH

Document processing, embeddings, AI reports, and notifications must not hold open HTTP requests.

Use:

```
API
 ↓
Queue
 ↓
Worker
 ↓
Database/status update
```

23.6 Schema evolution — HIGH

Assessments, scoring, AI output, subscriptions, and knowledge metadata will evolve.

Use versioning where historical correctness matters.

---

24. Security Checklist

Before production:

- [ ] Password hashing
- [ ] Secure sessions/JWT
- [ ] RBAC
- [ ] Tenant authorization
- [ ] Business-level data isolation
- [ ] Input validation
- [ ] File type validation
- [ ] File size limits
- [ ] Malware/security scanning where appropriate
- [ ] Rate limiting
- [ ] CSRF protection where applicable
- [ ] Secure CORS
- [ ] Secure environment variables
- [ ] Secret rotation process
- [ ] Audit logs
- [ ] Database backups
- [ ] Restore testing
- [ ] Encryption for sensitive data where required
- [ ] AI prompt-injection defenses for retrieved documents
- [ ] Authorization checks on AI tools
- [ ] Signed/controlled file URLs
- [ ] No cross-tenant leakage in logs/errors

---

25. Analytics

Track at minimum:

```
signup
onboarding_started
onboarding_completed
ai_question_asked
ai_recommendation_generated
assignment_created
assignment_completed
daily_checkin_completed
health_assessment_completed
kpi_added
sop_created
checklist_completed
resource_viewed
user_returned
```

Core product metrics:

```
Activation
AI Engagement
Daily Engagement
Execution
7-day retention
30-day retention
Business health improvement
Free → Paid conversion
```

---

26. MVP Definition of Done

A new business owner must be able to:

1. Create an account.
2. Complete onboarding.
3. See a personalized dashboard.
4. Complete daily check-in.
5. Receive a business score.
6. Complete a health assessment.
7. Receive an AI-generated report.
8. Ask the AI a business problem.
9. Receive methodology-based recommendations.
10. Convert a recommendation into an action.
11. Create and complete an assignment.
12. Track basic KPIs.
13. Access relevant resources.
14. View progress over time.
15. Admin can manage users, content, assessments, and knowledge base.

---

27. Recommended Development Order

Do not develop modules simply in UI/navigation order.

Build according to dependency:

```
1. Infrastructure
2. Database + tenancy
3. Authentication + authorization
4. Business onboarding
5. Core business data
6. AI provider abstraction
7. AI context architecture
8. Knowledge ingestion
9. Embeddings + pgvector
10. AI retrieval
11. AI Advisor tools
12. Daily score
13. Health assessment
14. Action plans
15. Assignments
16. KPI
17. Library
18. Admin
19. Analytics
20. Security hardening
21. Performance
22. End-to-end QA
```

---

28. Developer Team Estimate

Solo developer

Best when:

- developer owns architecture
- product decisions are stable
- UI scope is controlled

Expected MVP:

16–24 weeks

2 developers

Suggested split:

Developer A

- backend
- database
- AI
- RAG
- jobs

Developer B

- frontend
- dashboard
- onboarding
- admin
- UX

Expected MVP:

10–16 weeks

3 developers

Suggested:

- Backend/platform
- AI/data
- Frontend

Expected MVP:

8–14 weeks

Do not assume adding developers linearly reduces time. Architecture, product decisions, code review, QA, and integration create coordination overhead.

---

29. Testing Strategy

Unit tests

Test:

- scoring calculations
- permission checks
- entitlement checks
- tenant filters
- KPI calculations
- assignment state transitions

Integration tests

Test:

- auth
- business isolation
- AI tool authorization
- database workflows
- file ingestion
- embedding pipeline
- RAG retrieval

AI evaluation tests

Create a fixed dataset of business questions with expected relevant knowledge.

Measure:

- retrieval relevance
- hallucination rate
- methodology adherence
- correct tool selection
- action confirmation behavior
- answer quality
- token usage

- latency

End-to-end tests

Test the complete loop:

```
Onboard
 ↓
Ask problem
 ↓
Retrieve methodology
 ↓
AI recommendation
 ↓
Confirm action
 ↓
Assignment
 ↓
Complete assignment
 ↓
Dashboard reflects outcome
```

---

30. Performance Targets

Set initial targets rather than optimizing blindly.

Suggested MVP targets:

- Normal CRUD API: <500 ms at typical load
- Dashboard: <1.5 s excluding slow external AI calls
- AI first response: target <5–10 s depending on model/tool flow
- File upload acknowledgement: immediate; process asynchronously
- Embedding ingestion: asynchronous
- Background reports: asynchronous
- Search: target <500 ms for normal KB queries
- No request should remain open waiting for long document processing

Measure real production performance before tightening these targets.

---

31. Operational Requirements

Implement:

- structured logging
- request IDs
- AI request IDs
- job IDs
- error tracking
- database monitoring
- worker monitoring
- AI usage monitoring
- storage monitoring
- backup monitoring

- alerting for failed jobs

Every important asynchronous process should have a visible status:

```
queued
processing
completed
failed
retrying
```

---

32. Recommended Cost-Control Budget

For the development phase, use this budget model:

Minimum

$50/month

Comfortable

$150/month

Aggressive testing

$300/month

Extreme development/testing ceiling

$500/month

If the project consistently exceeds the $500/month testing ceiling before real users, stop and analyze usage rather than simply increasing the budget.

---

33. What Should Be Built First Technically

The first production-quality foundation should be:

```
PostgreSQL
+
Multi-tenancy
+
Auth/RBAC
+
Business model
+
AI provider abstraction
+
AI context service
+
Knowledge document/chunk model
+
Object storage
+
Background job system
+
Audit logging
+
Usage/cost tracking
```

This foundation will support almost every later module.

---

34. Final Recommendation

Do **not** optimize for building the maximum number of features.

Optimize for proving this loop:

```
Business Context
      ↓
AI Diagnosis
      ↓
Relevant Methodology
      ↓
Recommendation
      ↓
Action
      ↓
Assignment
      ↓
Completion
      ↓
Business Data
      ↓
Next Recommendation
```

The most technically important investments are:

1. Multi-tenant architecture
2. AI context selection
3. Knowledge retrieval/RAG
4. Proprietary methodology layer
5. AI tool architecture
6. Subscription entitlements
7. Background jobs
8. Security
9. AI cost tracking
10. Analytics

Everything else can be incrementally added around this core.

---

35. Final Planning Numbers

Area	Estimate
MVP — solo developer	**16–24 weeks**
MVP — 2 developers	**10–16 weeks**
MVP — 3 developers	**8–14 weeks**
Phase 2	**6–12 weeks**
Phase 3	**3–6 months**
Phase 4	**6–18+ months**
Lean testing infrastructure	**$20–75/mo**
Heavy testing	**$100–300/mo**
Extreme testing ceiling	**$300–500/mo**
Primary DB	**PostgreSQL**
Vector search	**pgvector**
Semantic embeddings	**Embedding model/API**
File storage	**Object storage**
Long-running work	**Background workers/queue**

> **Important:** These are engineering planning estimates. Vendor prices, AI model prices, traffic, user behavior, storage volume, and implementation quality can materially change actual costs.

---

36. Claude Planning Instructions

When generating the final implementation plan from this document:

1. Treat the supplied requirements as the product scope baseline.
2. Keep the MVP strictly limited to Priority 1 unless a dependency requires otherwise.
3. Clearly separate MVP, Phase 2, Phase 3, and Phase 4.
4. For every feature provide:
   - purpose
   - dependencies
   - frontend work
   - backend work
   - database work
   - AI work
   - background-job work
   - security considerations
   - testing requirements

   - complexity
   - estimated engineering time
5. Produce a dependency-aware development sequence.
6. Provide low/base/high cost scenarios.
7. Make AI cost assumptions explicit.
8. Include token-budget and AI-cost-control architecture.
9. Include PostgreSQL + pgvector architecture and explain that embeddings are generated by an embedding model, not pgvector.
10. Include multi-tenant authorization requirements in every business-owned module.
11. Do not recommend MongoDB unless a specific requirement demonstrates a concrete advantage.
12. Do not add ERP/CRM/WhatsApp features to MVP.
13. Identify risky areas separately from straightforward CRUD.
14. Provide acceptance criteria for each major milestone.
15. Include a final go/no-go checklist for MVP launch.
16. Distinguish estimates from confirmed requirements.
17. Do not assume a large engineering team.
18. Optimize for maintainability and future expansion rather than premature complexity.